



COURSE CODE/TITLE: CPE 010

SECTION: CPE21S2

DATE PERFORMED: July 17, 2026

DATE SUBMITTED: July 17, 2026

NAME: Johrel Louie Silao.

INSTRUCTOR: Engr. Jimlord M. Quejado

ACTIVITY NO. 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot

```
1  #include <iostream>
2  #include <string.h>
3
4  class Student {
5      private:
6          std::string studentName;
7          int studentAge;
8          int *data;
9          size_t size;
10
11     public:
12         //Constructor
13         Student (std::string newName = "John Doe", int newAge = 18) {
14             studentName = std::move (newName);
15             studentAge = newAge;
16             std::cout << "Constructor Successfully Called." << std::endl;
17         };
18
19         //Destructor
20         ~Student () {
21             std::cout << "Destructor Successfully Called." << std::endl;
22         }
23
24         //Copy Constructor
25         Student (const Student &copyStudent) {
26             studentName = copyStudent.studentName;
27             studentAge = copyStudent.studentAge;
28             std::cout << "Copy Constructor Successfully Called." << std::endl;
29         }
30     }
```

```

32 //Copy Assignment Operator
33 Student& operator = (const Student &copyStudent) {
34     if (this != &copyStudent) {
35         studentName = copyStudent.studentName;
36         studentAge = copyStudent.studentAge;
37         std::cout << "Copy Assignment Operator Successfully Called." << std::endl;
38     }
39     return *this;
40 }
41
42 //Display Attributes
43 void printDetails () {
44     std::cout << this->studentName << " " << this->studentAge << std::endl;
45 }
46
47 };
48
49 int main() {
50     Student student1("Roman", 28);
51     Student student2(student1);
52     Student student3;
53     student3 = student2;
54     |
55     return 0;
56 }

```

```

Constructor Successfully Called.
Copy Constructor Successfully Called.
Constructor Successfully Called.
Copy Assignment Operator Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.

```

Observation

Observing the output of the program, using the first int main()function, it will first create a new Student object named "Student1", which will call the Constructor successfully. It will then create a new object (Student2) which is a copy of Student1 which calls the Copy Constructor successfully then it will create Student3, which will call the Constructor again. Lastly, it will call the Copy Assignment Operator, since there is no object that was created, it will just copy Student2. Now it will delete all 3 of the new constructors.

Table 2-1. Initial Driver Program

Screenshot

```

1  #include <iostream>
2  #include <string.h>
3
4  class Student {
5      private:
6          std::string studentName;
7          int studentAge;
8          int *data;
9          size_t size;
10
11     public:
12         //Constructor
13         Student (std::string newName = "John Doe", int newAge = 18) {
14             studentName = std::move (newName);
15             studentAge = newAge;
16             std::cout << "Constructor Successfully Called." << std::endl;
17
18         };
19
20         //Destructor
21         ~Student () {
22             std::cout << "Destructor Successfully Called." << std::endl;
23         }
24
25         //Copy Constructor
26         Student (const Student &copyStudent) {
27             studentName = copyStudent.studentName;
28             studentAge = copyStudent.studentAge;
29             std::cout << "Copy Constructor Successfully Called." << std::endl;
30         }
31
32         //Copy Assignment Operator
33         Student& operator = (const Student &copyStudent) {
34             if (this != &copyStudent) {
35                 studentName = copyStudent.studentName;
36                 studentAge = copyStudent.studentAge;
37                 std::cout << "Copy Assignment Operator Successfully Called." << std::endl;
38             }
39             return *this;
40         }
41
42         //Display Attributes
43         void printDetails () {
44             std::cout << this->studentName << " " << this->studentAge << std::endl;
45         }
46
47     };
48
49     int main() {
50         const size_t j = 5;
51
52         Student studentList[j] = {};
53         std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
54         int ageList[j] = {15, 16, 18, 19, 16};
55
56         return 0;
57     }

```

	<pre> Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. </pre>
Observation	Observing the program, it first initializes an array of 5 Student objects, since there are no arguments involved, it will just immediately display “Constructor Successfully Called.” With this, it will now reach the end of the code block to delete all of the Student objects that was created, which will display “Destructor Successfully Called.”

Table 2-2. Modified Driver Program with Student Lists

Loop A	<pre> 1 #include <iostream> 2 #include <string.h> 3 4 class Student { 5 private: 6 std::string studentName; 7 int studentAge; 8 int *data; 9 size_t size; 10 11 public: 12 //Constructor 13 Student (std::string newName = "John Doe", int newAge = 18) { 14 studentName = std::move (newName); 15 studentAge = newAge; 16 std::cout << "Constructor Successfully Called." << std::endl; 17 }; 18 19 //Destructor 20 ~Student () { 21 std::cout << "Destructor Successfully Called." << std::endl; 22 } 23 24 //Copy Constructor 25 Student (const Student &copyStudent) { 26 studentName = copyStudent.studentName; 27 studentAge = copyStudent.studentAge; 28 std::cout << "Copy Constructor Successfully Called." << std::endl; 29 } 30 </pre>
--------	---

```

32 //Copy Assignment Operator
33 Student& operator = (const Student &copyStudent) {
34     if (this != &copyStudent) {
35         studentName = copyStudent.studentName;
36         studentAge = copyStudent.studentAge;
37         std::cout << "Copy Assignment Operator Successfully Called." << std::endl;
38     }
39     return *this;
40 }
41
42 //Display Attributes
43 void printDetails () {
44     std::cout << this->studentName << " " << this->studentAge << std::endl;
45 }
46
47 };
48
49 int main() {
50     const size_t j = 5;
51
52     Student studentList[j] = {};
53     std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
54     int ageList[j] = {15, 16, 18, 19, 16};
55
56     for(int i = 0; i < j; i++){ //loop A
57         Student *ptr = new Student(namesList[i], ageList[i]);
58         studentList[i] = *ptr;
59
60         delete ptr;
61         ptr = nullptr;
62     }
63
64     return 0;
65 }

```

```

49 int main() {
50     const size_t j = 5;
51
52     Student studentList[j] = {};
53     std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
54     int ageList[j] = {15, 16, 18, 19, 16};
55
56     for(int i = 0; i < j; i++){ //loop A
57         Student *ptr = new Student(namesList[i], ageList[i]);
58         studentList[i] = *ptr;
59
60         delete ptr;
61         ptr = nullptr;
62     }

```

	<pre> Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. </pre>
Observation	It will first create 5 Student objects and it will allocate a heap student with the real name and age and copies it into the array, then it will delete the heap student. With this, it will loop for 5 times. Since the details are not called, the data inside is still inside the memory but it was never displayed. Then it will now be deleted because the Destructor was called successfully.

Loop B

```
1  #include <iostream>
2  #include <string.h>
3
4  class Student {
5      private:
6          std::string studentName;
7          int studentAge;
8          int *data;
9          size_t size;
10
11     public:
12         //Constructor
13         Student (std::string newName = "John Doe", int newAge = 18) {
14             studentName = std::move (newName);
15             studentAge = newAge;
16             std::cout << "Constructor Successfully Called." << std::endl;
17         };
18
19         //Destructor
20         ~Student () {
21             std::cout << "Destructor Successfully Called." << std::endl;
22         }
23
24         //Copy Constructor
25         Student (const Student &copyStudent) {
26             studentName = copyStudent.studentName;
27             studentAge = copyStudent.studentAge;
28             std::cout << "Copy Constructor Successfully Called." << std::endl;
29         }
30     }
```

```

32 //Copy Assignment Operator
33 Student& operator = (const Student &copyStudent) {
34     if (this != &copyStudent) {
35         studentName = copyStudent.studentName;
36         studentAge = copyStudent.studentAge;
37         std::cout << "Copy Assignment Operator Successfully Called." << std::endl;
38     }
39     return *this;
40 }
41
42 //Display Attributes
43 void printDetails () {
44     std::cout << this->studentName << " " << this->studentAge << std::endl;
45 }
46
47 };
48
49 int main() {
50     const size_t j = 5;
51
52     Student studentList[j] = {};
53     std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
54     int ageList[j] = {15, 16, 18, 19, 16};
55
56     for(int i = 0; i < j; i++){ //loop A
57         Student *ptr = new Student(namesList[i], ageList[i]);
58         studentList[i] = *ptr;
59
60         delete ptr;
61         ptr = nullptr;
62     }
63
64     return 0;
65 }

```

```

49 int main() {
50     const size_t j = 5;
51
52     Student studentList[j] = {};
53     std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
54     int ageList[j] = {15, 16, 18, 19, 16};
55
56     for(int i = 0; i < j; i++){ //loop A
57         Student *ptr = new Student(namesList[i], ageList[i]);
58         studentList[i] = *ptr;
59
60         delete ptr;
61         ptr = nullptr;
62     }

```


	<pre> Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. John Doe 18 John Doe 18 John Doe 18 John Doe 18 John Doe 18 Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. </pre>
Observation	It will first create 5 Student objects and it will allocate a heap student with the real name and age and copies it into the array, then it will delete the heap student. With this, it will loop for 5 times. Since the details are called, the data will be printed. Then it will now be deleted because the Destructor was called successfully.

Output	<pre> Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Carly 15 Freddy 16 Sam 18 Zack 19 Cody 16 Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called. </pre>
Observation	Since

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, destructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

Problem 5: Discuss the

```

1  #include <iostream>
2  #include <string>
3  #include <iomanip>
4
5  class Fruit {
6  private:
7      std::string name;
8      double price;
9      int quantity;
10
11 public:
12     // Constructor
13
14     Fruit(std::string n = " ", double p = 0, int q = 0) {
15         name = n;
16         price = p;
17         quantity = q;
18     }
19
20     // Destructor
21
22     ~Fruit() {
23         std::cout << "Fruit Destructor is Called: " << name << std::endl;
24     }
25
26     // Copy Constructor
27
28     Fruit(const Fruit& other) {
29         name = other.name;
30         price = other.price;
31         quantity = other.quantity;
32     }

```

Figure 1.1 Code Snippet: Fruit Class Definition

Analysis: Defines the Fruit class that contains the private members: name, price and quantity. A constructor with default arguments, destructor and a copy constructor.

```

34 // Copy Assignment
35
36 Fruit& operator=(const Fruit& other) {
37     if (this != &other) {
38         name = other.name;
39         price = other.price;
40         quantity = other.quantity;
41     }
42     return *this;
43 }
44
45 double totalSum() const {
46     return price * quantity;
47 }
48
49 void receipt() const {
50     std::cout << std::left
51         << std::setw(15) << "Fruit"
52         << std::setw(15) << name
53         << std::setw(12) << price
54         << std::setw(12) << quantity
55         << std::setw(12) << totalSum()
56         << std::endl;
57 }
58 };

```

Figure 1.2 Code Snippet: Fruit Class Assignment Operator

Analysis: Implements the copy assignment operator using self-assignment checks to copy the member data. It also has a totalSum() function to calculate the costs of the grocery and also a receipt() using formatted I/O for a formatted table output.

```
60 class Vegetable {
61 private:
62     std::string name;
63     double price;
64     int quantity;
65
66 public:
67     // Constructor
68
69     Vegetable(std::string n = " ", double p = 0, int q = 0) {
70         name = n;
71         price = p;
72         quantity = q;
73     }
74
75     // Destructor
76
77     ~Vegetable() {
78         std::cout << "Vegetable Destructor is Called: " << name << std::endl;
79     }
80
81     // Copy Constructor
82
83     Vegetable(const Vegetable& other) {
84         name = other.name;
85         price = other.price;
86         quantity = other.quantity;
87     }
```

Figure 1.3 Code Snippet: Vegetable Class Definition and Copy Constructor

Analysis: Defining the Vegetable class with its attributes, a parameter constructor, destructor and a copy constructor..

```

89      // Copy Assignment
90
91      Vegetable& operator=(const Vegetable& other) {
92          if (this != &other) {
93              name = other.name;
94              price = other.price;
95              quantity = other.quantity;
96          }
97          return *this;
98      }
99
100     double totalSum() const {
101         return price * quantity;
102     }
103
104     void receipt() const {
105         std::cout << std::left
106             << std::setw(15) << "Vegetables"
107             << std::setw(15) << name
108             << std::setw(12) << price
109             << std::setw(12) << quantity
110             << std::setw(12) << totalSum()
111             << std::endl;
112     }
113 };
114
115 double totalSum(Fruit* fruits[], int fruitCount,
116               Vegetable* vegetables[], int vegetableCount);
117

```

Figure 1.4 Code Snippet: Vegetable Class Methods and Function Prototypes

Analysis: Completing the Vegetable class implementation with a copy assignment operator with the totalSum(), and receipt() methods

```

118 int main() {
119
120     Fruit* fruitsTaken[2];
121     Vegetable* vegetablesTaken[2];
122
123     fruitsTaken[0] = new Fruit("Apple", 10, 7);
124     fruitsTaken[1] = new Fruit("Banana", 10, 8);
125
126     vegetablesTaken[0] = new Vegetable("Broccoli", 60, 12);
127     vegetablesTaken[1] = new Vegetable("Lettuce", 50, 10);
128
129     std::cout << "\n=====MEMORY=====\n";
130
131     std::cout << "Apple Pointer      : " << fruitsTaken[0] << std::endl;
132
133     std::cout << "Banana Pointer      : " << fruitsTaken[1] << std::endl;
134
135     std::cout << "Broccoli Pointer   : " << vegetablesTaken[0] << std::endl;
136
137     std::cout << "Lettuce Pointer    : " << vegetablesTaken[1] << std::endl;
138
139     std::cout << "-----\n";
140     std::cout << "                      RECEIPT\n";
141     std::cout << "-----\n";
142
143     std::cout << std::left
144               << std::setw(15) << "Type"
145               << std::setw(15) << "Item"
146               << std::setw(10) << "Price"
147               << std::setw(10) << "Quantity"
148               << std::setw(10) << "Total"
149               << std::endl;
150     std::cout << "-----\n";

```

Figure 1.5 Code Snippet: Dynamic Allocation and Address Logging

Analysis: Shows how the dynamic memory allocation inside the main() by instantiating the Fruit and Vegetable objects on the heap using the “new” operator and stores their memory address the array of pointers.

```

152     std::cout << "Fruits:" << std::endl;
153     for (int i = 0; i < 2; i++) {
154         fruitsTaken[i]->receipt();
155     }
156
157     std::cout << "Vegetables:" << std::endl;
158     for (int i = 0; i < 2; i++) {
159         vegetablesTaken[i]->receipt();
160     }
161
162     std::cout << "\nTotal Amount in Pesos: "
163         << totalSum(fruitsTaken, 2, vegetablesTaken, 2)
164         << std::endl;
165
166     char answer;
167
168     std::cout << "\nDelete Lettuce? (Y/N): ";
169     std::cin >> answer;
170
171     if (answer == 'Y' || answer == 'y') {
172
173         std::cout << "\n-----Pointer List Before Deletion-----\n";
174
175         std::cout << "Apple Pointer    : " << fruitsTaken[0] << std::endl;
176         std::cout << "Banana Pointer   : " << fruitsTaken[1] << std::endl;
177         std::cout << "Broccoli Pointer : " << vegetablesTaken[0] << std::endl;
178         std::cout << "Lettuce Pointer  : " << vegetablesTaken[1] << std::endl;
179
180         std::cout << "-----\n";

```

Figure 1.6 Code Snippet: Receipt and Deletion of Item

Analysis: Using pointers to apply the member methods using loops to display the items in the receipt and its total price. Additionally, it asks the user to delete an item.

```

181
182     std::cout << "\nDeleting Lettuce...\n";
183
184     delete vegetablesTaken[1];
185
186     vegetablesTaken[1] = nullptr;
187
188     std::cout << "\nMemory successfully deallocated.\n";
189
190     std::cout << "\n-----Pointer List After Deletion-----\n";
191
192     std::cout << "Apple Pointer    : " << fruitsTaken[0] << std::endl;
193     std::cout << "Banana Pointer   : " << fruitsTaken[1] << std::endl;
194     std::cout << "Broccoli Pointer : " << vegetablesTaken[0] << std::endl;
195     std::cout << "Lettuce Pointer  : " << vegetablesTaken[1] << std::endl;
196
197     std::cout << "-----\n";
198
199     if (vegetablesTaken[1] == nullptr) {
200         std::cout << "\nLettuce successfully deleted.\n";
201         std::cout << "Lettuce pointer is now nullptr.\n";
202     }
203
204     else {
205         std::cout << "\nDeletion cancelled.\n";
206     }
207
208     std::cout << "\nPress Enter to Continue...";
209     std::cin.ignore();
210     std::cin.get();

```

Figure 1.7 Code Snippet: Memory Deallocation and Null Pointer

Analysis: Shows a dynamic reallocation using “delete” on the targeted object pointer and resetting the null pointer to prevent any pointer bugs.

```

212     std::cout << "-----\n";
213     std::cout << "                Updated Grocery Receipt\n";
214     std::cout << "-----\n";
215
216     std::cout << "Fruits:" << std::endl;
217     for (int i = 0; i < 2; i++) {
218         if (fruitsTaken[i] != nullptr) {
219             fruitsTaken[i]->receipt();
220         }
221     }
222
223     std::cout << "\nVegetables:" << std::endl;
224     for (int i = 0; i < 2; i++) {
225         if (vegetablesTaken[i] != nullptr) {
226             vegetablesTaken[i]->receipt();
227         }
228     }
229
230     std::cout << "\nNew Total Amount in Pesos: "
231             << totalSum(fruitsTaken, 2, vegetablesTaken, 2)
232             << std::endl;
233
234     std::cout << "\n-----Pointer List After Cleanup-----\n";
235
236     std::cout << "Apple Pointer   : " << fruitsTaken[0] << std::endl;
237     std::cout << "Banana Pointer  : " << fruitsTaken[1] << std::endl;
238     std::cout << "Broccoli Pointer: " << vegetablesTaken[0] << std::endl;
239     std::cout << "Lettuce Pointer : " << vegetablesTaken[1] << std::endl;
240
241     std::cout << "-----\n";

```

Figure 1.8 Code Snippet: Conditional Receipt

Analysis: Navigates objects arrays using nullptr before calling methods.

```

264
265     if (fruitsTaken[0] == nullptr &&
266         fruitsTaken[1] == nullptr &&
267         vegetablesTaken[0] == nullptr &&
268         vegetablesTaken[1] == nullptr) {
269
270         std::cout << "\nAll dynamically allocated memory has been deallocated successfully.\n";
271         std::cout << "No memory leaks remain.\n";
272     }
273
274     return 0;
275 }
276
277 double totalSum(Fruit* fruits[], int fruitCount,
278               Vegetable* vegetables[], int vegetableCount) {
279
280     double total = 0;
281
282     for (int i = 0; i < fruitCount; i++) {
283         if (fruits[i] != nullptr) {
284             total += fruits[i]->totalSum();
285         }
286     }
287
288     for (int i = 0; i < vegetableCount; i++) {
289         if (vegetables[i] != nullptr) {
290             total += vegetables[i]->totalSum();
291         }
292     }
293
294     return total;
295 }

```

Figure 1.9 Code Snippet: Memory Cleanup

Analysis: It shows the implementation of the totalSum() function to safely calculate the total costs and the program will validate complete memory cleanup with no memory leaks.

```
=====MEMORY=====
Apple Pointer   : 0x101401ad0
Banana Pointer  : 0x101401900
Broccoli Pointer: 0x101401930
Lettuce Pointer : 0x101401960

=====RECEIPT=====
Type      Item      Price  Quantity  Total
-----
Fruits:
Fruit     Apple     10      7         70
Fruit     Banana     10      8         80
Vegetables:
Vegetables Broccoli    60     12        720
Vegetables Lettuce     50     10        500

Total Amount in Pesos: 1370

Delete Lettuce? (Y/N): Y

-----Pointer List Before Deletion-----
Apple Pointer   : 0x101401ad0
Banana Pointer  : 0x101401900
Broccoli Pointer: 0x101401930
Lettuce Pointer : 0x101401960

Deleting Lettuce...
Vegetable Destructor is Called: Lettuce

Memory successfully deallocated.

-----Pointer List After Deletion-----
Apple Pointer   : 0x101401ad0
Banana Pointer  : 0x101401900
Broccoli Pointer: 0x101401930
Lettuce Pointer : 0x0

Lettuce successfully deleted.
Lettuce pointer is now nullptr.

Press Enter to Continue...cd "/Users/lourdes/Desktop/CPE010_SIL
Supplementary_Activity && "/Users/lourdes/Desktop/CPE010_SIL
```

Figure 2.1 Output Snippet: First Allocation, Receipt and Deletion of Item

Analysis: Outputs the initial heap memory address that is assigned to each and every item and displays the receipt format and the execution of the Vegetable destructor when lettuce is deleted when the user inputted.

```

-----
Updated Grocery Receipt
-----
Fruits:
Fruit      Apple      10      7      70
Fruit      Banana     10      8      80

Vegetables:
Vegetables Broccoli   60     12     720

New Total Amount in Pesos: 870

-----Pointer List After Cleanup-----
Apple Pointer   : 0x101401ad0
Banana Pointer  : 0x101401900
Broccoli Pointer: 0x101401930
Lettuce Pointer : 0x0

Cleaning up remaining memory...
Fruit Destructor is Called: Apple
Fruit Destructor is Called: Banana
Vegetable Destructor is Called: Broccoli

Memory successfully cleaned up.

-----Pointer List After Cleanup-----
Apple Pointer   : 0x0
Banana Pointer  : 0x0
Broccoli Pointer: 0x0
Lettuce Pointer : 0x0

All dynamically allocated memory has been deallocated successfully.
No memory leaks remain.

```

Figure 2.2 Output Snippet: Successful Deletion and Memory Cleanup

Analysis: Displays the updated receipt and also the deleted item, while also demonstrating the complete program termination cleanup where all of the dynamic items are deleted, destructors called, and memory pointers set to 0x0 without memory leaks.

```

-----
Updated Grocery Receipt
-----
Fruits:
Fruit      Apple      10      7      70
Fruit      Banana     10      8      80

Vegetables:
Vegetables Broccoli   60      12     720
Vegetables Lettuce    50      10     500

New Total Amount in Pesos: 1370

-----Pointer List After Cleanup-----
Apple Pointer   : 0x1037c9ad0
Banana Pointer  : 0x1037c9900
Broccoli Pointer: 0x1037c9930
Lettuce Pointer : 0x1037c9960
-----

Cleaning up remaining memory...
Fruit Destructor is Called: Apple
Fruit Destructor is Called: Banana
Vegetable Destructor is Called: Broccoli

Memory successfully cleaned up.

-----Pointer List After Cleanup-----
Apple Pointer   : 0x0
Banana Pointer  : 0x0
Broccoli Pointer: 0x0
Lettuce Pointer : 0x1037c9960
-----

```

Figure 2.3 Output Snippet: Cancelling Management and Remaining Pointer

Analysis: Shows the action when the memory cleanup happens when an item pointer is left active. While the other items are cleaned up and any pointer that is not cleared will continue to point to its assigned addresses.

```
=====MEMORY=====
Apple Pointer      : 0x1037c9ad0
Banana Pointer     : 0x1037c9900
Broccoli Pointer   : 0x1037c9930
Lettuce Pointer    : 0x1037c9960
=====

                        RECEIPT
=====
Type      Item      Price  Quantity  Total
=====
Fruits:
Fruit     Apple      10      7         70
Fruit     Banana     10      8         80
Vegetables:
Vegetables Broccoli   60     12        720
Vegetables Lettuce    50     10        500

Total Amount in Pesos: 1370

Delete Lettuce? (Y/N): n

Deletion cancelled.

Press Enter to Continue...
```

Figure 2.4 Output Snippet: User Cancellation Output

Analysis: Shows how the user input handles the item deletion and the program preserves all allocated object addresses and displays the final receipt without applying any destructors.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of



COMPUTER
ENGINEERING

QUEZON CITY

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.